MAPREDUCE LABORATORY WORKSHEET | MINI PROJECT

Smart Campus Analytics using Python Hadoop Streaming

Completed worksheet, results, interpretation and execution evidence

Student name	**Deepesh Kakkar**
Roll number	**102315286**
Date of submission	**18 October 2026**

Contents

1. Project summary 2. Student implementation worksheet (Q1-Q15) 3. Expected output worksheet 4. Result interpretation 5. Performance considerations 6. Extension challenges (incl. Job 7) 7. Viva questions 8. Execution evidence

1. Project summary

SmartCampusAnalytics analyses four campus datasets (Wi-Fi logs, canteen transactions, library visits and transport records) with the six required MapReduce jobs, written as Python mapper and reducer scripts for Hadoop Streaming, plus a seventh job for the extension challenge that chains Job 3 and Job 4. All datasets are **synthetic** (seed 42); no real student data is used.

Dataset	Data rows	Valid	Rejected (deliberately malformed)
`wifi_logs.csv`	5,005	5,000	5
`canteen.csv`	3,179 (1,500 transactions)	3,174	5
`library.csv`	1,505	1,500	5
`transport.csv`	3,568	3,563	5

Verification	Result
Local simulation	All seven jobs run; 22/22 validation invariants pass; every output row matches an independent reference implementation.
Automated tests	38 tests run, 38 passed, 0 failed, 0 skipped.
Real Hadoop	Hadoop 3.5.0 (standalone mode, OpenJDK 17): all ten stages completed successfully and all seven final outputs are identical to the local results (first run 11 October 2026 for Jobs 1-6; re-run 18 October 2026 with Job 7).

2. Student implementation worksheet

Q1. For Job 1, what should the Mapper output key be? Why?

Key: the hour bucket `YYYY-MM-DD-HH`, e.g. `2026-08-10-09`. MapReduce groups records by key, so the key must be the unit we aggregate over: one calendar hour. The date is included so that 09:00 on different days stays separate.

Q2. What is the Mapper output value? Can you design a more efficient representation?

Value: `DataMB,DurationMin,1`. The trailing 1 is the session count, so the reducer computes total data, total duration and sessions in one pass.

More efficient: aggregate before the shuffle with a **combiner** (or in-mapper combining) so each map task emits one (sum, sum, count) record per hour instead of one per session. Measured on Hadoop 3.5.0: the combiner reduced Job 1's 5,000 map records to 112. A binary format such as Avro would shrink the bytes further.

Q3. Why is the hour extracted from the timestamp instead of using the complete timestamp as the key?

A full timestamp is almost unique per record, so every group would contain about one record and nothing would be aggregated. Truncating to the hour produces the grouping granularity the question asks for: totals per hour.

Q4. Which operations can be performed using a Combiner?

Operations that are associative and commutative and whose output has the same format as the reducer input: sums and counts. Used here for Jobs 1, 2, 3, 4 (stage 2), 5 (stage 1) and 6. An average can only be combined as a (sum, count) pair, never as an average of averages. Not combinable: Job 4 stage 1 (needs the whole transaction) and the Job 5 filter (needs the global average).

Q5. For Job 2, define the key and value emitted by the Mapper.

Key: `AccessPoint` (e.g. `AP03`). Value: `DataMB,DurationMin,1`. The reducer outputs TotalDataMB, TotalDurationMin, Sessions and AverageSessionDuration = TotalDurationMin / Sessions.

Q6. For Job 4, explain how you will generate every unique pair of products within a transaction.

Stage 1 mapper emits `TransactionID -> Item`, so the shuffle delivers every item of one transaction to one reducer call, regardless of input order. The reducer takes the distinct items, sorts them, and emits each combination of two once (`itertools.combinations(sorted(set(items)), 2)`): n distinct items give n(n-1)/2 pairs, with no self-pairs and no duplicates. Stage 2 sums the pairs across all transactions.

Q7. Why must the product pair be sorted, e.g., (Coffee, Sandwich) rather than sometimes (Sandwich, Coffee)?

Both orders describe the same pair. If both appeared, MapReduce would treat them as two different keys, splitting one count into two smaller wrong counts. Sorting gives every pair one canonical key.

Q8. For Job 5, how will you compute the overall average using more than one MapReduce job?

Stage 1: key StudentID, output `TotalDuration,Visits` per student. Stage 2 (global): every record is mapped to one key `GLOBAL`; a single reducer sums duration and visits and outputs OverallAverage = total / visits (visit-weighted). Stage 3 (filter): reads the global line as a side file (Hadoop distributed cache) and flags students whose average is strictly greater than 1.5 x OverallAverage.

Separate jobs share no memory, so the global result must be passed as data (a file), not as a variable.

Q9. For Job 6, explain why RouteID and Hour form a composite key.

Congestion belongs to one route at one time. Keying by route alone mixes rush hour with quiet hours; keying by hour alone mixes all routes. The composite key `RouteID|YYYY-MM-DD-HH` groups exactly the observations whose average speed we need.

Q10. Which jobs require more than one MapReduce stage? Justify your answer.

Job 4 (two stages): pairs must be formed per transaction (grouping by TransactionID) before they can be counted across transactions (grouping by pair). **Job 5** (three stages): the filter needs the overall average, which only exists after all records are aggregated. Jobs 1, 2, 3 and 6 each need only one grouping, so one stage suffices. The extension Job 7 is a third kind of chaining: a map-only job that consumes the outputs of Jobs 3 and 4.

Q11. What happens if a reducer receives millions of values for one key?

The reducer still works, because it streams values with O(1) memory. But that reducer becomes a straggler: the whole job waits for it, and its input is one very large shuffle transfer. Remedies: a combiner to shrink the values, or salting the hot key (key#1..key#n) and merging in a second stage.

Q12. Explain how data skew could affect the Smart Campus application.

Some keys are far larger than others: lunch-time hours (Jobs 1 and 3), popular access points (Job 2), best-selling items such as Tea and Samosa (Jobs 3 and 4), and busy routes (Job 6). With hash partitioning, the reducers that own these keys run longest and delay the job. Combiners reduce the effect for the additive jobs. Job 4 stage 1 skews if one transaction has very many items.

Q13. Design a custom partitioning strategy for Job 6.

Use `KeyFieldBasedPartitioner` with key-field separator `|` and option `-k1,1`, so records are partitioned by RouteID only. All hours of a route go to the same reducer, giving per-route, time-ordered output. It is correct because a (route, hour) group never spans two reducers. If one route is much busier, partition on the full key instead for better balance.

```
hadoop jar hadoop-streaming.jar \
  -D mapreduce.map.output.key.field.separator='|' \
  -D mapreduce.partition.keypartitioner.options=-k1,1 \
  -files ... -input ... -output ... -mapper ... -reducer ... \
  -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

Q14. What changes are necessary if the input contains malformed records?

Parse with a real CSV parser (quoted commas), validate every field in the mapper (field count, timestamp format, numeric and non-negative values), skip invalid rows instead of crashing, and count them by reason with Hadoop counters so nothing is lost silently. Implemented here: 5 malformed rows per dataset are rejected and reported (e.g. wifi.rejected.bad_timestamp=1), and records = valid + rejected is verified.

Q15. Suggest one additional campus analytics problem that can be solved using MapReduce.

Classroom occupancy: count distinct students per access point per hour from Wi-Fi logs (stage 1: key (AP, hour, student) to remove duplicates; stage 2: count per (AP, hour)) to find over-crowded or under-used rooms.

3. Expected output worksheet

Job	Expected key	Expected value	Student result
Job 1 - Peak Wi-Fi	`YYYY-MM-DD-HH`	TotalDataMB, TotalDurationMin, Sessions	112 hours. Peak hour **2026-08-10 14:00**: 18,491.90 MB, 5,827 min, 100 sessions.
Job 2 - Access Point	`AccessPoint`	TotalDataMB, TotalDurationMin, Sessions, AvgDuration	12 access points; 394-440 sessions each; average session 57.99-62.76 min.
Job 3 - Canteen	`Item`	TotalQuantity, TotalRevenue	12 items. Highest revenue: Sandwich (Rs. 47,120). Most units: Tea (651).
Job 4 - Co-Purchase	`(ItemA, ItemB)`	TransactionCount	66 pairs. Top: (Samosa, Tea) 242; (Coffee, Sandwich) 220.
Job 5 - Library	`StudentID` / `GLOBAL`	TotalDuration, Visits / OverallAverage	Overall average 62.38 min (93,568 min / 1,500 visits); threshold 93.57 min; 6 anomalous students: S104, S132, S157, S189, S193, S285.
Job 6 - Transport	`RouteID|Hour`	AverageSpeed, ObservationCount, Status	784 route-hours; 86 CONGESTED (average below 20 km/h).
Job 7 - Revenue pairs (extension)	`(ItemA, ItemB)` (map-only)	TransactionCount, UnitPriceA, UnitPriceB, EstPairRevenue	66 pairs. Top: (Coffee, Sandwich) Rs. 25,300; (Samosa, Tea) Rs. 8,470.

4. Result interpretation

Job 1. Wi-Fi demand peaks in late morning and early afternoon; summed over the week, hours 11, 14 and 15 carry the most data. The busiest single hour is 2026-08-10 14:00 (18,491.90 MB). Network capacity should be planned for these peaks, not for the daily average.

Job 2. Load is spread fairly evenly across the 12 access points (sessions within about 12%, data within about 17%), with an average session of about 60 minutes. No access point is overloaded in this data; in real data the same job would reveal hotspots.

Job 3. Tea (651 units) and Samosa (618) sell the most units, but Sandwich earns the most revenue (Rs. 47,120) because of its higher price. Stock planning should follow quantity; pricing and promotion decisions should follow revenue.

Job 4. Two combinations dominate: Samosa + Tea (242 transactions) and Coffee + Sandwich (220). Every other pair appears in at most 39 transactions. These two pairs are natural meal-deal candidates.

Job 5. 6 of 200 students average more than 93.57 minutes per visit (1.5 x the overall 62.38). Five have repeated long visits (about 206-251 minutes on average); S157 is flagged from a single 98-minute visit, which is weak evidence. Flags indicate unusual usage worth reviewing, not misconduct.

Job 6. 86 of 784 route-hours are congested. Congestion clusters in the rush hours 08:00-09:59 and 17:00-18:59 and on routes R03 (37 route-hours) and R02 (35). Average speed below 20 km/h is an indicator only; it cannot distinguish traffic from scheduled stops.

Job 7 (extension). Frequency and value rank pairs differently. Samosa + Tea is the most frequent pair (242 transactions) but earns only about Rs. 8,470 at average prices; Coffee + Sandwich (220 transactions) earns about Rs. 25,300, roughly three times more. Rare pairs with Veg Biryani or Masala Dosa (25-36 transactions) fill places 3-8 by value. A meal deal on Coffee + Sandwich protects the most revenue; one on Samosa + Tea mainly drives footfall.

5. Performance considerations

The jobs are limited by data movement, not computation: every map output record is serialised, sorted and transferred to a reducer during the shuffle. Reducing shuffle volume is therefore the most effective optimisation. Combiners on the additive jobs do this: on Hadoop, Job 1's 5,000 map records became 112 after combining, and in a 1,000,000-row local benchmark one map task's shuffle fell from 249,981 records to 119. Keeping values as (sum, count) makes averages combinable. Composite keys and partitioners decide how evenly reducers are loaded; skewed keys such as lunch hours, popular routes or very large transactions create stragglers. Job 4's pair generation grows quadratically with basket size, so basket size, not row count, drives its cost. Multi-stage jobs (4 and 5) pay extra job start-up and intermediate writes, so extra stages are used only where a different grouping is genuinely required.

6. Extension challenges

All eight extension challenges from section 12 of the worksheet are implemented.

Challenge	Implementation and evidence
Combiner for the Wi-Fi job	`src/sum_combiner.py`, an element-wise sum that is safe for every additive job. On Hadoop it reduced Job 1's 5,000 map records to 112.
Top three hours	`outputs/job1_peak_hours.csv`: 2026-08-10 14:00, 2026-08-13 11:00, 2026-08-11 15:00.
Custom partitioner for transport	KeyFieldBasedPartitioner on RouteID (Q13), verified on Hadoop 3.5.0.
Structured serialization	Job 4 uses JSON pair keys such as `["Coffee", "Sandwich"]`, so item names containing commas or quotes stay unambiguous.
Chain Job 3 and Job 4: high-revenue pairs	**Job 7**, described below.
Count rejected malformed records	5 malformed rows per dataset (20 in total) are rejected by reason and reported as Hadoop counters; records = valid + rejected is checked.
With and without a Combiner	Identical outputs. Shuffle records fell from 5,000 to 112 on Hadoop (Job 1), and from 249,981 to 119 per map task in a 1,000,000-row local benchmark.
Multiple reducers	Identical outputs with 1, 3 and 4 reducers in the local simulator (Hadoop was run with one reducer).

Job 7: high-revenue item pairs

Job 7 joins the two finished outputs on the map side. Its input is the Job 4 output (`ItemA,ItemB,TransactionCount`); the Job 3 output (`Item,TotalQuantity,TotalRevenue`) reaches every mapper as a side file through the distributed cache. Each pair is independent, so the job is **map-only** (`-reducer NONE`): no shuffle, sort or reduce. The score is `EstPairRevenue = TransactionCount x (UnitPriceA + UnitPriceB)`, where UnitPrice = TotalRevenue / TotalQuantity: the revenue of the pair's transactions at one unit of each item at its average price.

```
hadoop jar hadoop-streaming.jar \
  -files job7_revenue_pairs_mapper.py,mr_common.py,out/job3/part-00000#item_stats.txt \
  -input out/job4 -output out/job7 \
  -mapper "python3 job7_revenue_pairs_mapper.py" -reducer NONE
```

Rank	Pair	Transactions	Est. pair revenue
1	Coffee + Sandwich	220	Rs. 25,300
2	Samosa + Tea	242	Rs. 8,470
3	Masala "Special" Dosa + Veg Biryani	27	Rs. 5,670
4	Sandwich + Veg Biryani	27	Rs. 5,400
5	Cookies + Veg Biryani	36	Rs. 5,220

Verified by the independent reference implementation (every one of the 66 rows matches), two new unit tests, and a real Hadoop 3.5.0 run on 18 October 2026 whose output matched the local result.

7. Viva questions

1. What are the three major phases of MapReduce?

Map (transform input records into key-value pairs), shuffle and sort (group all values of a key and deliver them sorted to one reducer), and reduce (aggregate each key's values into the output).

2. What is the difference between Mapper output and Reducer output?

Mapper output is intermediate: many key-value pairs per input record, local to the task, and shuffled to reducers. Reducer output is final: one aggregated record per key, written to the output directory (part files).

3. Why does Hadoop sort Mapper output before it reaches the Reducer?

So that all values of a key arrive next to each other. The reducer can then process each key in one streaming pass, detecting the key change, without holding all data in memory.

4. What is a Combiner and when is it safe to use one?

A local mini-reducer run on map output before the shuffle. It is safe when the operation is associative and commutative and its output format equals its input format (sums, counts, min, max), because Hadoop may run it zero, one or many times.

5. What is a Partitioner?

The function that decides which reducer receives each key, by default hash(key) mod number of reducers. A custom partitioner, such as RouteID-only for Job 6, controls load balance and output layout.

6. Can a MapReduce job have zero reducers?

Yes. A map-only job (reducers = 0) skips shuffle and sort and writes the mapper output directly, which is useful for filtering, joins with a small side file or format conversion. Job 7 in this project is map-only.

7. What happens when there are multiple reducers?

Keys are split into partitions, one per reducer, which run in parallel and each write a part file. Every key goes to exactly one reducer, so results stay correct; verified here with 1, 3 and 4 reducers giving identical output.

8. What is a composite key?

A key built from more than one field, used when grouping depends on a combination. Example: `RouteID|Hour` in Job 6.

9. How would you handle duplicate records?

Define what makes a record identical, key on that identity and keep one per key in a deduplication stage. In Job 4, duplicate item lines within a transaction are removed with a set before pairs are formed.

10. Why are multiple MapReduce jobs sometimes necessary?

One job provides one grouping. When a computation needs a second grouping or a global value, as in Job 4 (per transaction, then per pair) and Job 5 (per student, then global, then filter), jobs are chained, with one job's output as the next job's input.

11. What is data locality?

Running a map task on the node that already stores its input block, so that computation moves to the data instead of copying data over the network.

12. What is the effect of an excessively large key group?

One reducer gets most of the work and becomes a straggler, the job finishes only when it does, and its shuffle input may be very large. A combiner or key salting reduces the effect.

13. How can you reduce network traffic during shuffle?

Use a combiner, emit compact values ((sum, count) rather than raw records), filter and project early in the mapper, and compress map output (mapreduce.map.output.compress=true).

14. How would you test your Mapper independently?

Pipe a small known input into the script and inspect the output: `cat data/wifi_logs.csv | python3 src/job1_wifi_mapper.py`. This project also uses unit tests with hand-computed expected outputs for each mapper and reducer.

15. How would you modify the solution to work with millions of records?

Store the input in HDFS and run on a cluster with several reducers, enable combiners, compress intermediate output, and keep reducers streaming (O(1) memory per key). The scripts already stream; a 1,000,000-row Job 1 run completed locally in about 6 seconds.

8. Execution evidence

Figures 1-13 are real screenshots of the macOS Terminal, captured on 11 October 2026 while each command ran, before Job 7 was added; the test and check counts in them are those of that date. Original full-resolution images are in the `screenshots/` folder. Figure 14 is the console output of the 18 October 2026 Hadoop re-run with Job 7.

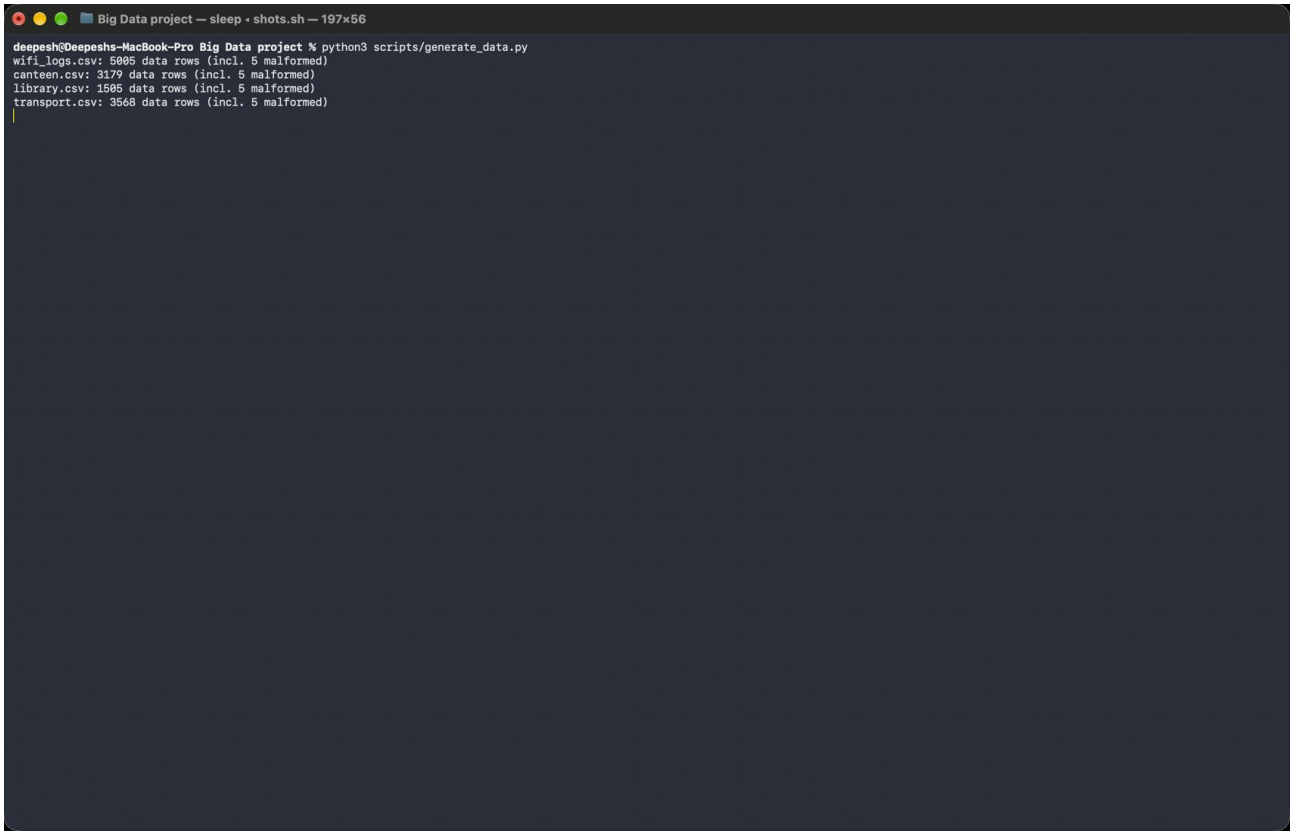


Figure 1. Generating the four synthetic datasets (seed 42).

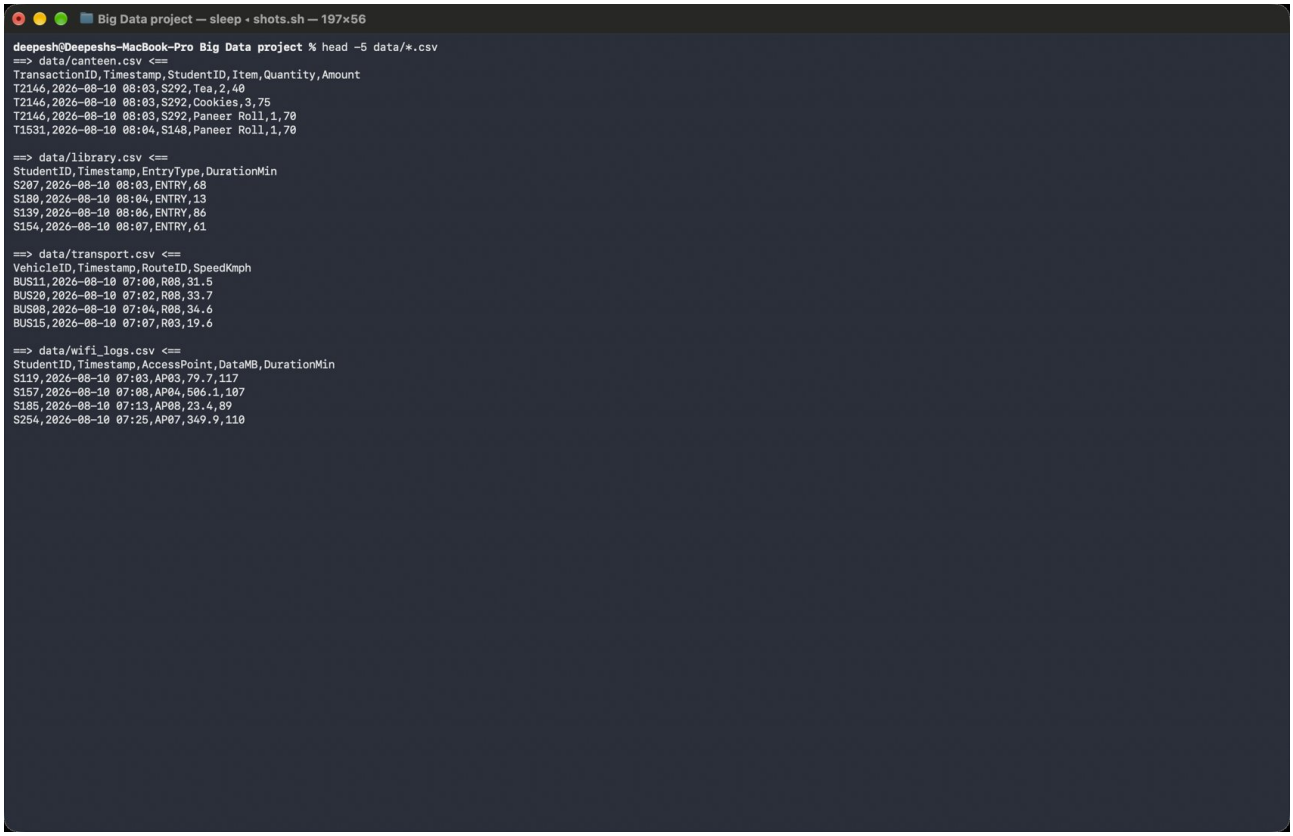


Figure 2. Headers and first rows of the four input datasets.

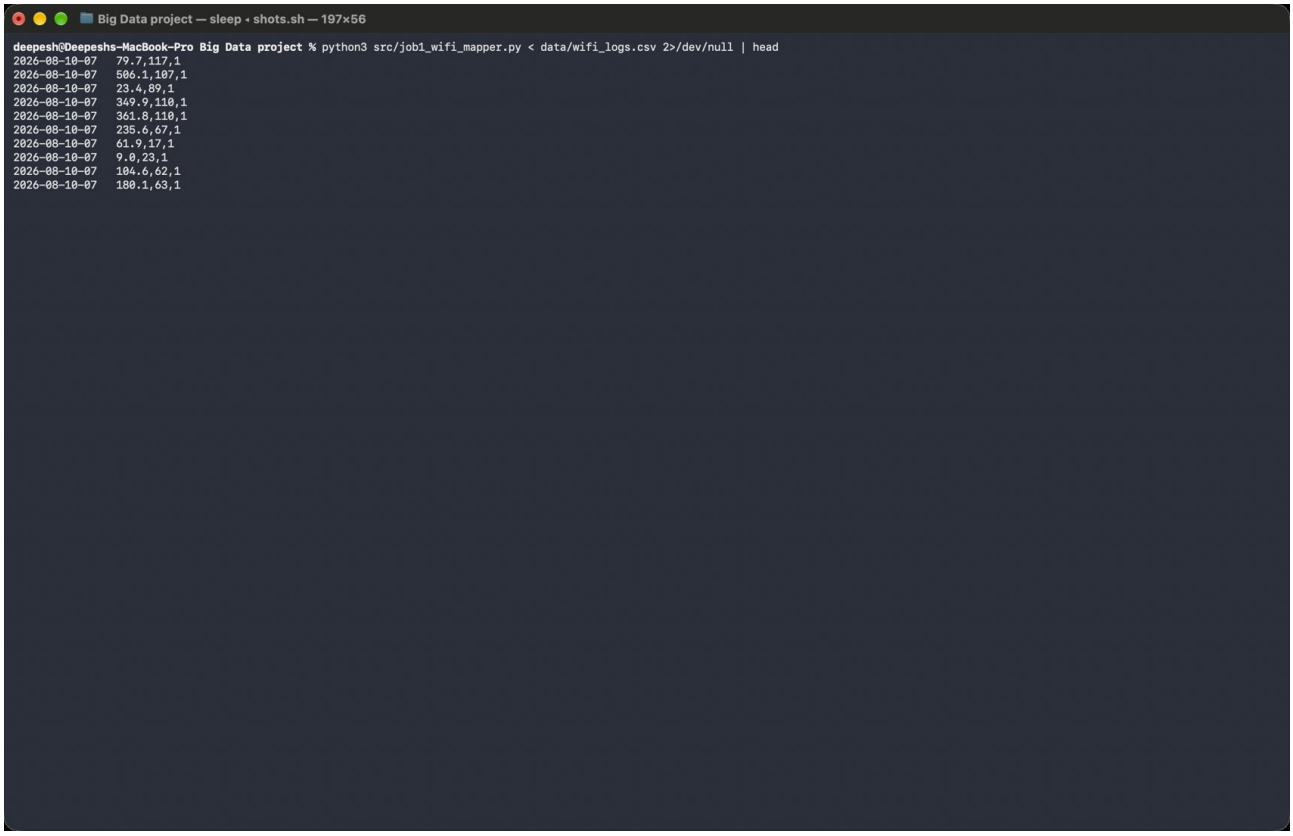


Figure 3. Job 1 mapper output: key YYYY-MM-DD-HH, value DataMB,DurationMin,1 (worksheet section 7, step 4).

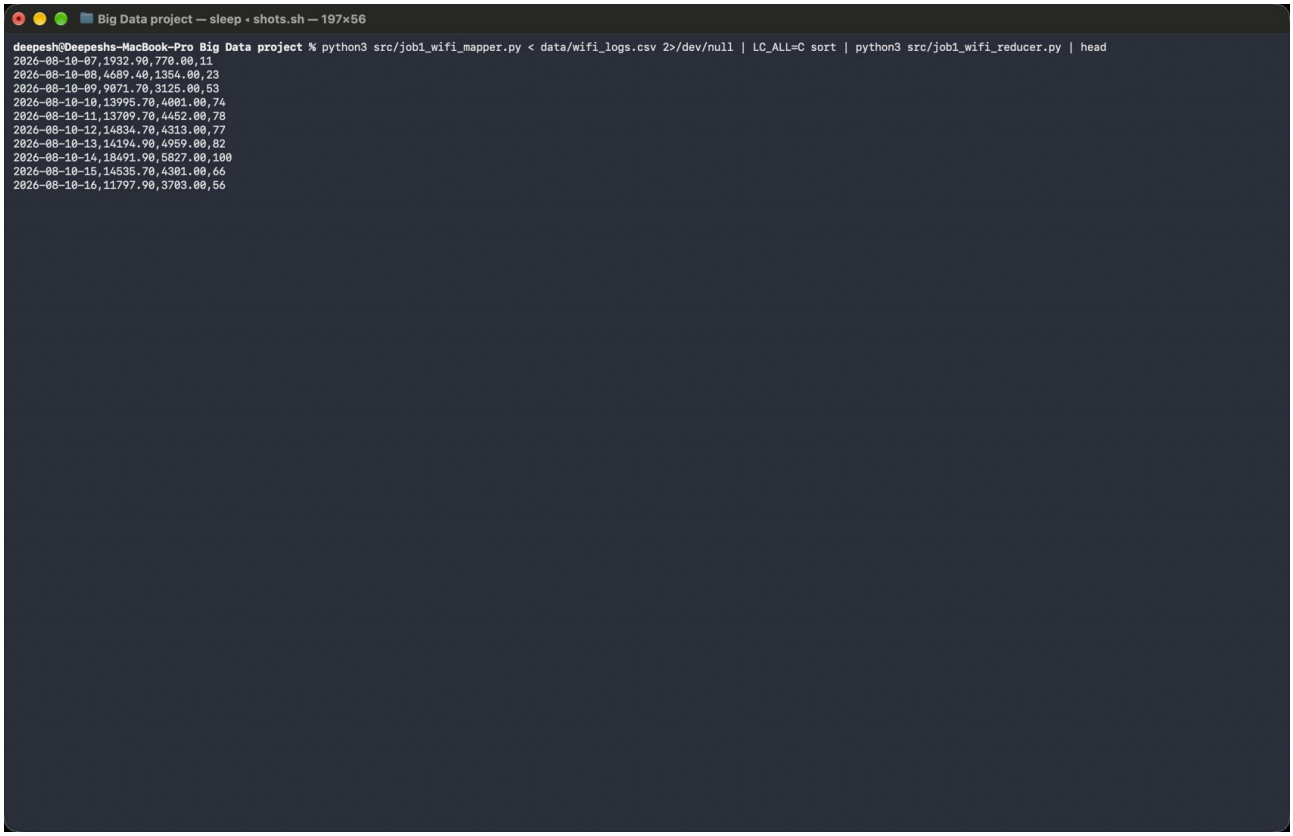


Figure 4. Job 1 mapper | sort | reducer: hourly totals (worksheet section 7, step 6).

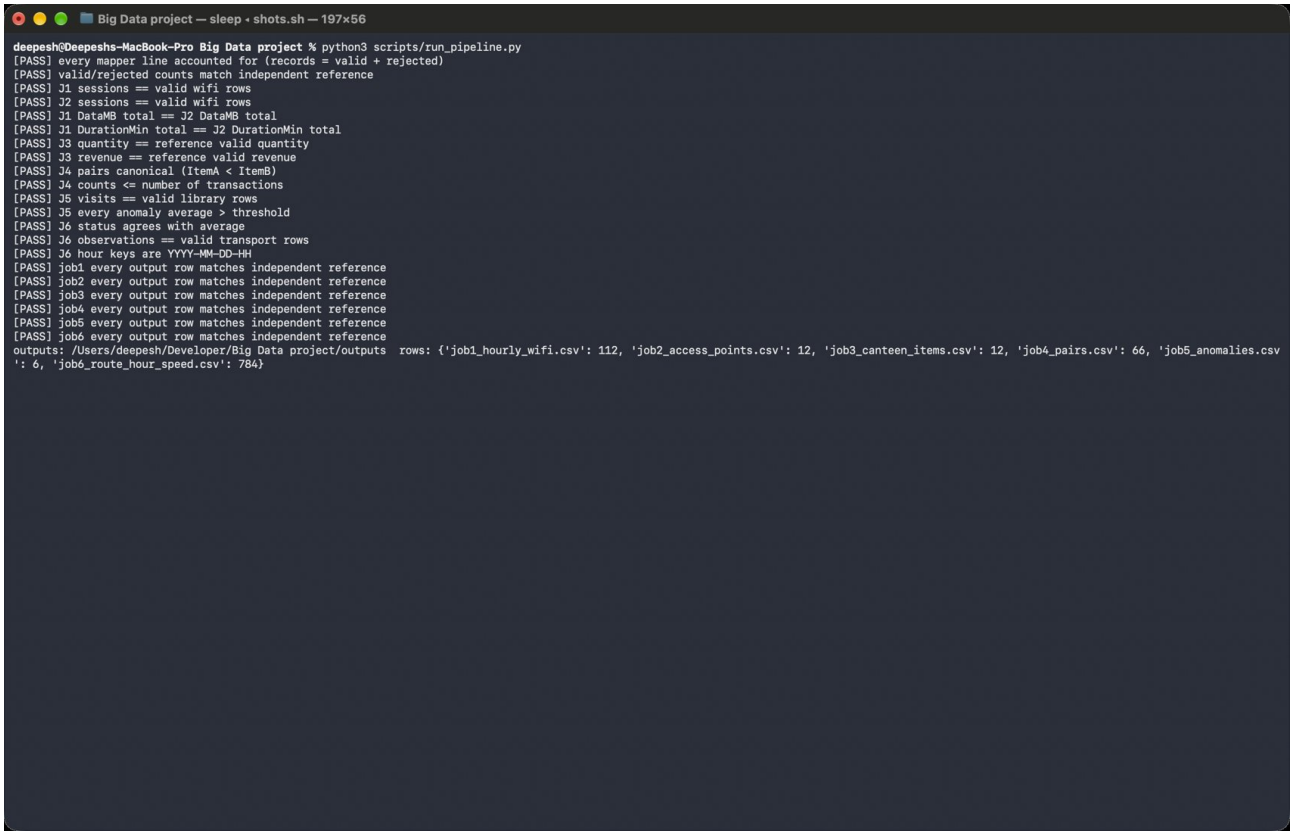


Figure 5. Local pipeline: all six jobs run and all 21 validation checks pass.

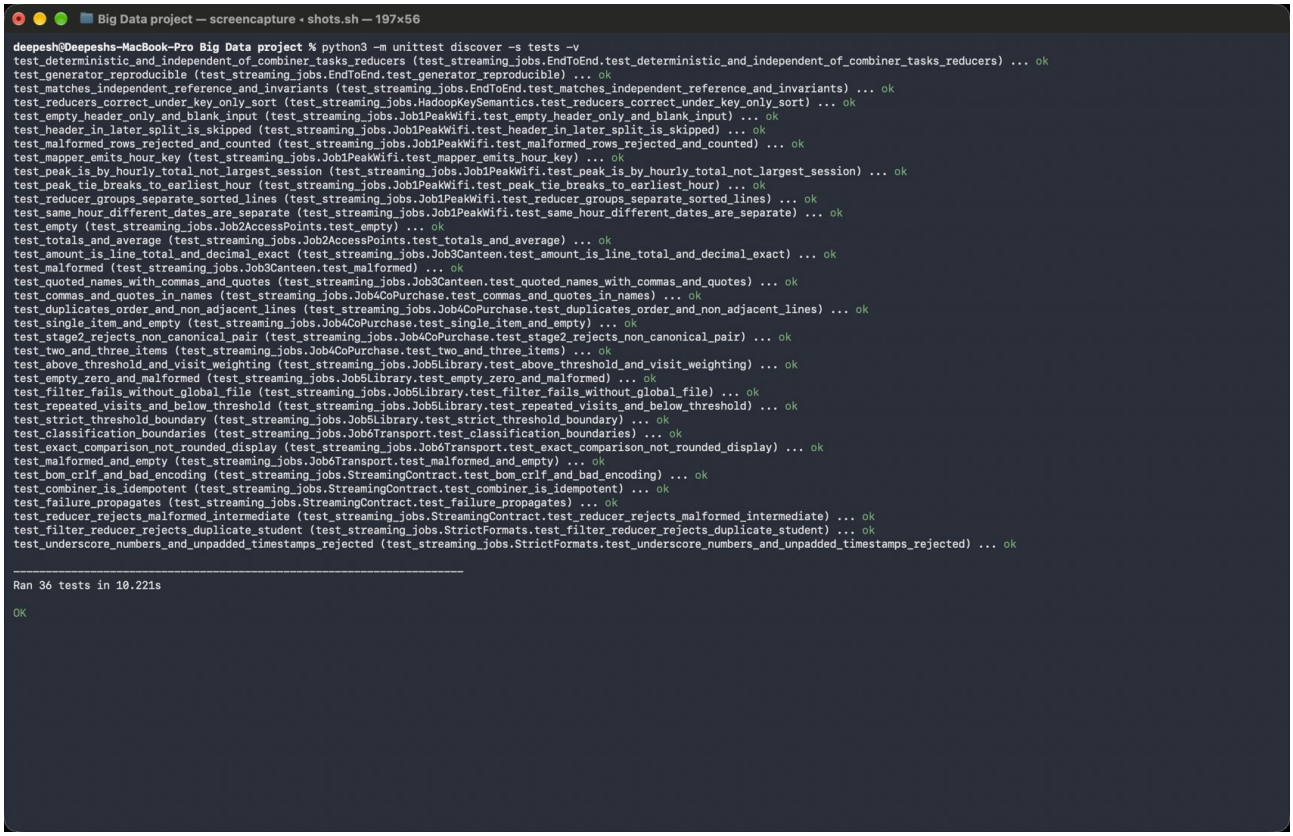


Figure 6. Automated test suite: 36 tests, all OK.

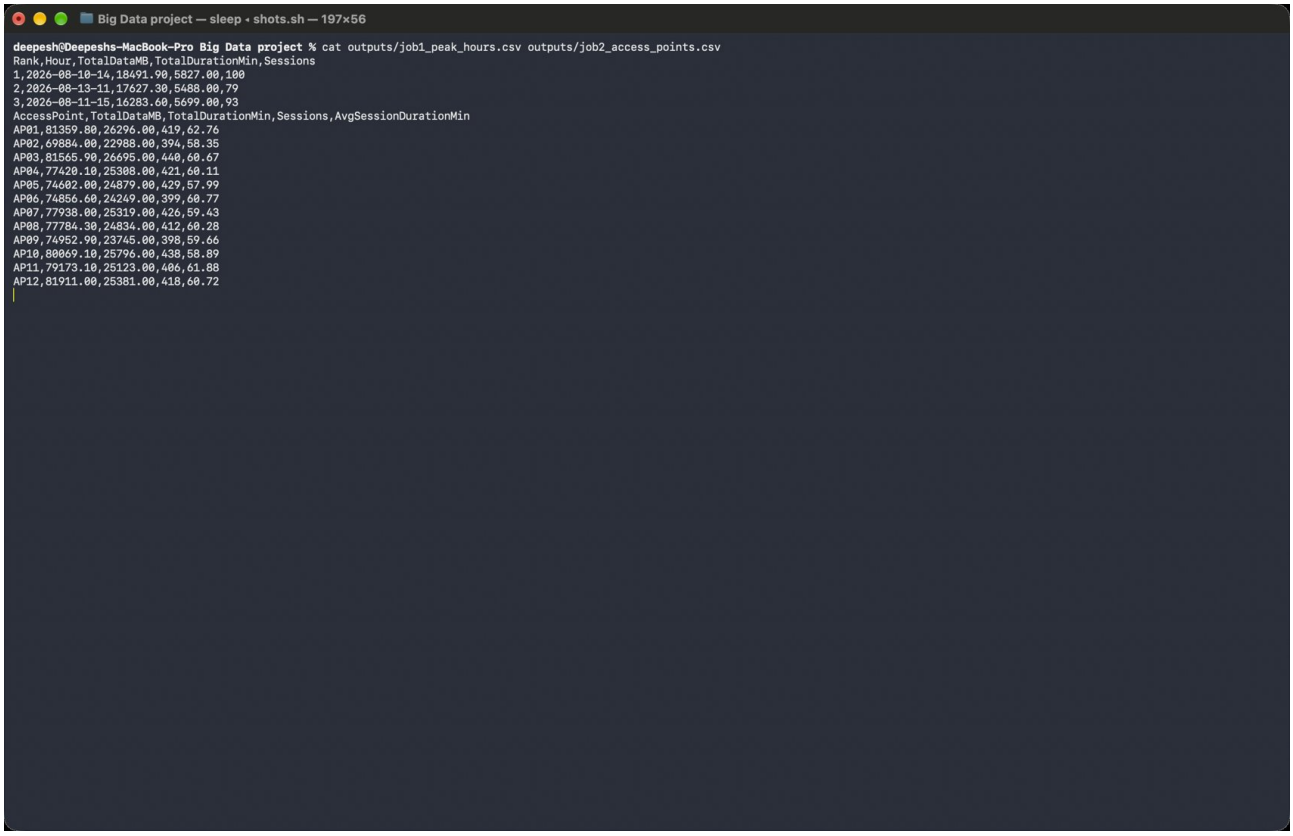


Figure 7. Job 1 top-3 peak hours and Job 2 access-point utilisation.

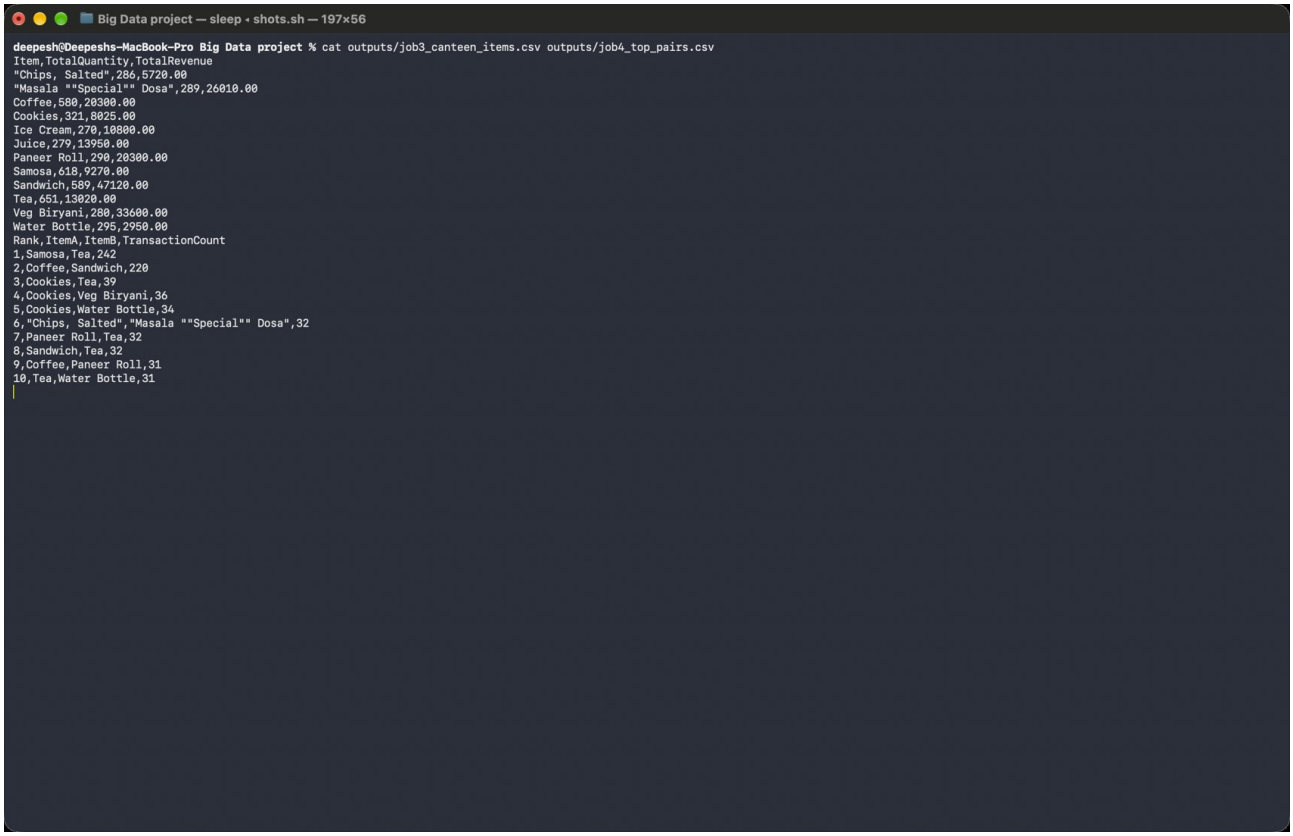


Figure 8. Job 3 item totals and Job 4 top co-purchased pairs.

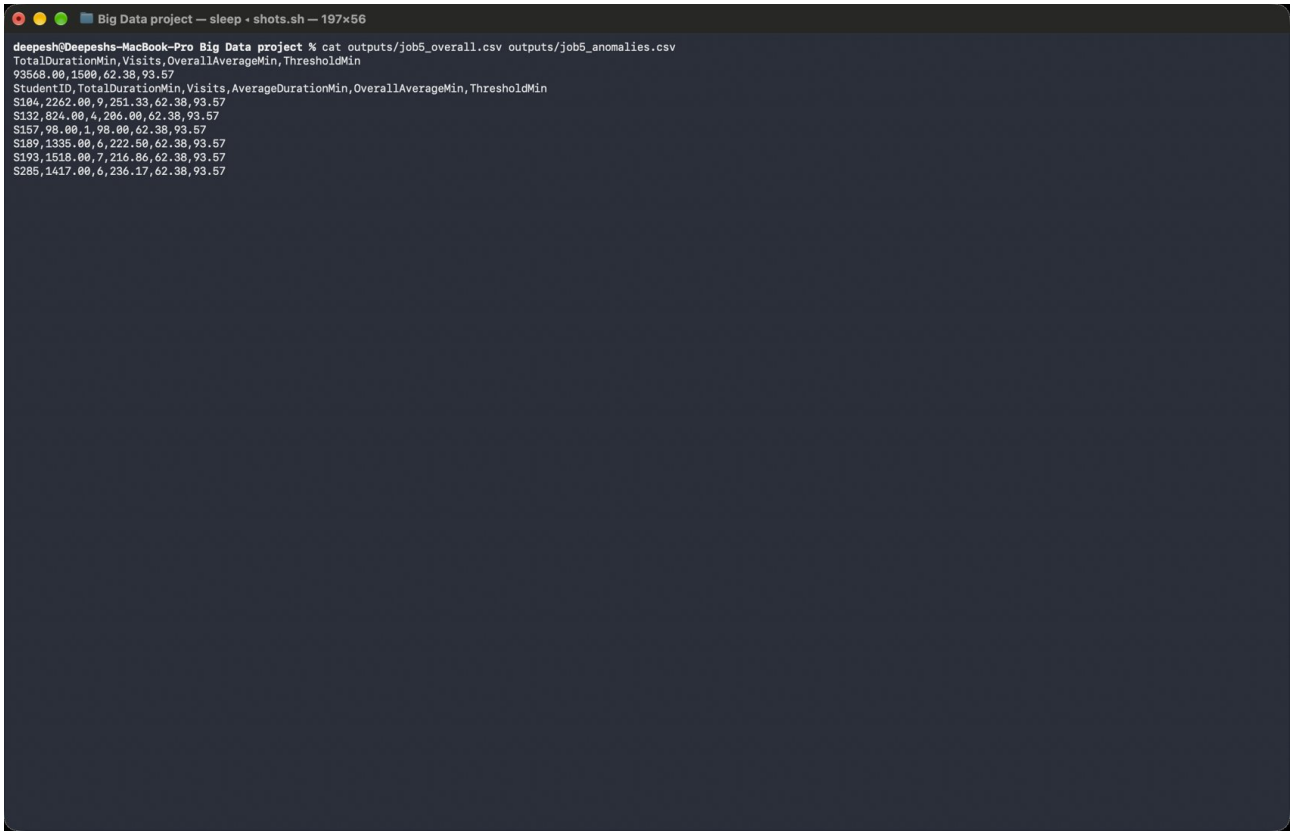


Figure 9. Job 5 overall average, threshold and anomalous students.

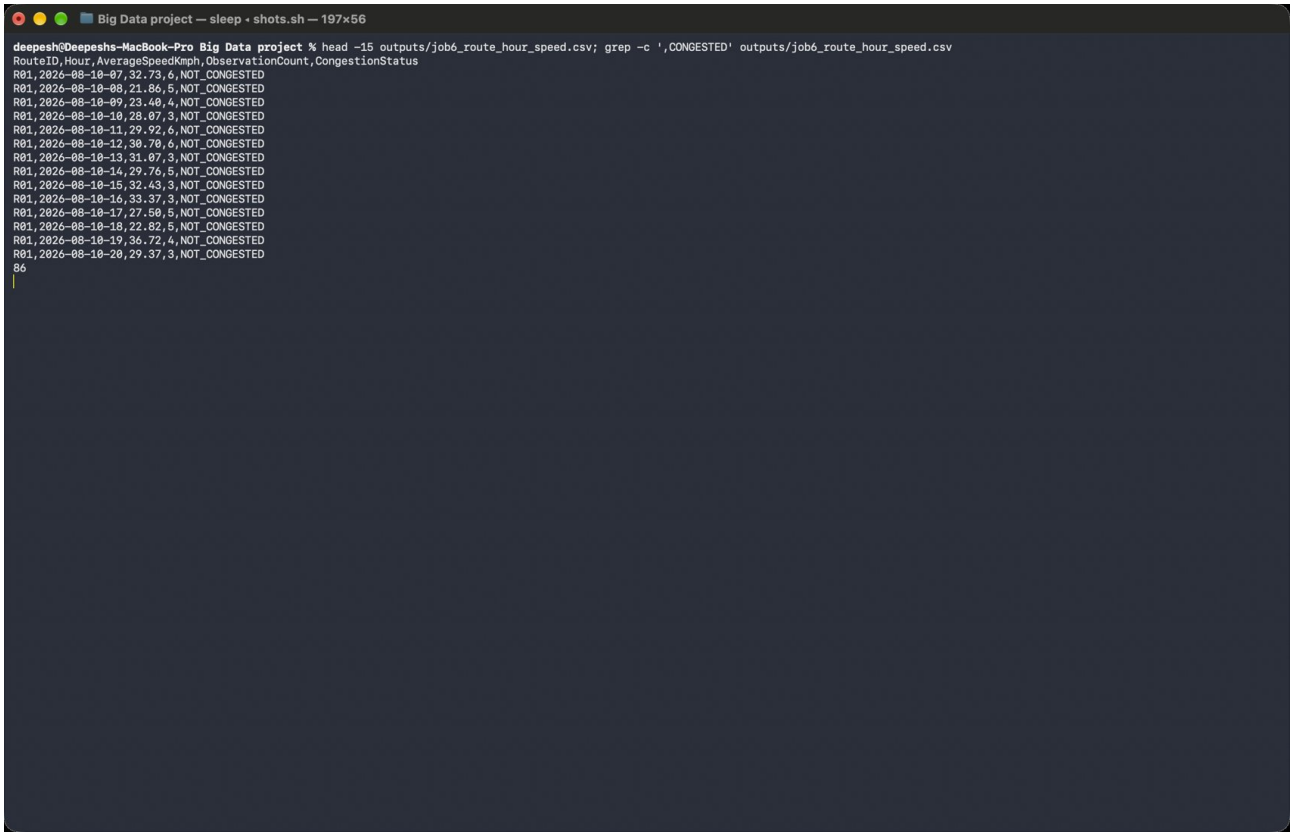


Figure 10. Job 6 route-hour average speeds and number of congested route-hours.

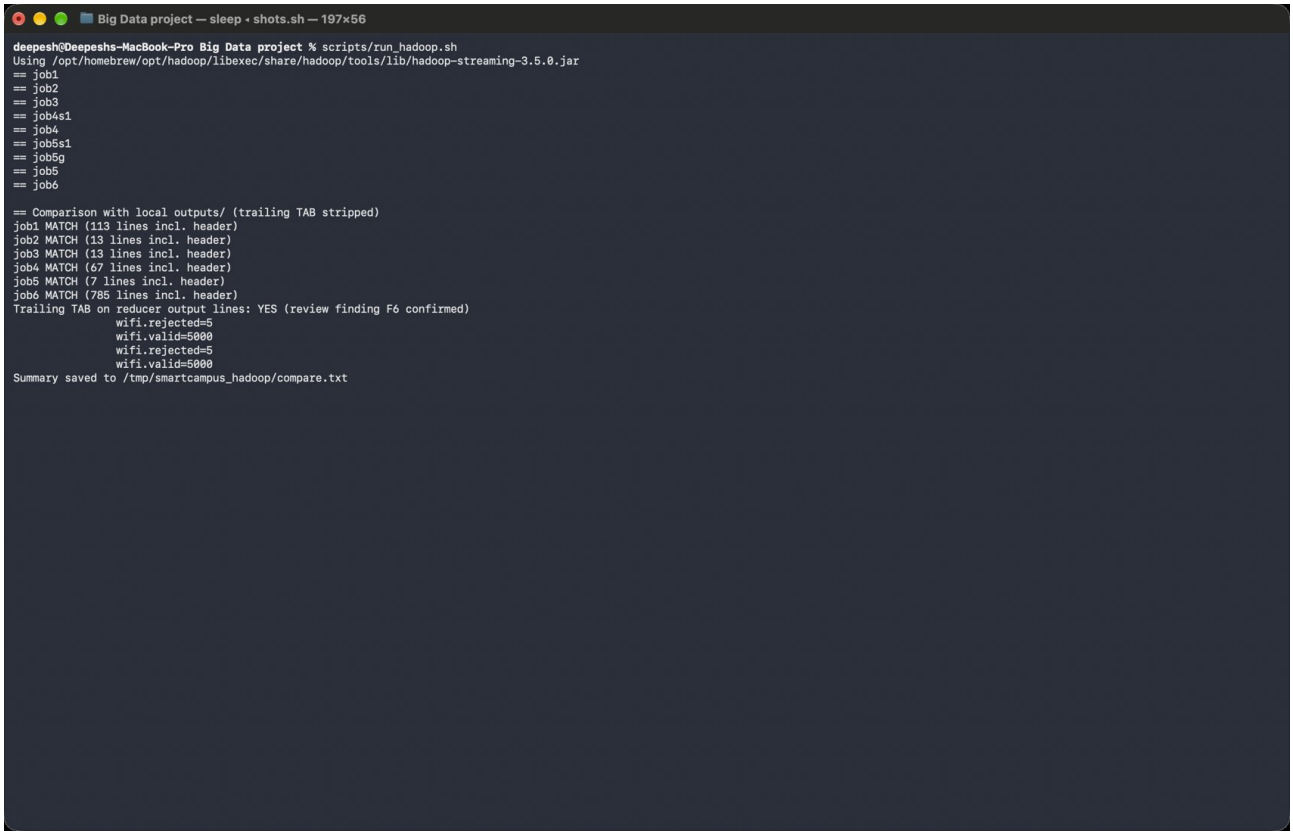


Figure 11. Real Hadoop 3.5.0 run of all nine stages: all six outputs MATCH the local results.

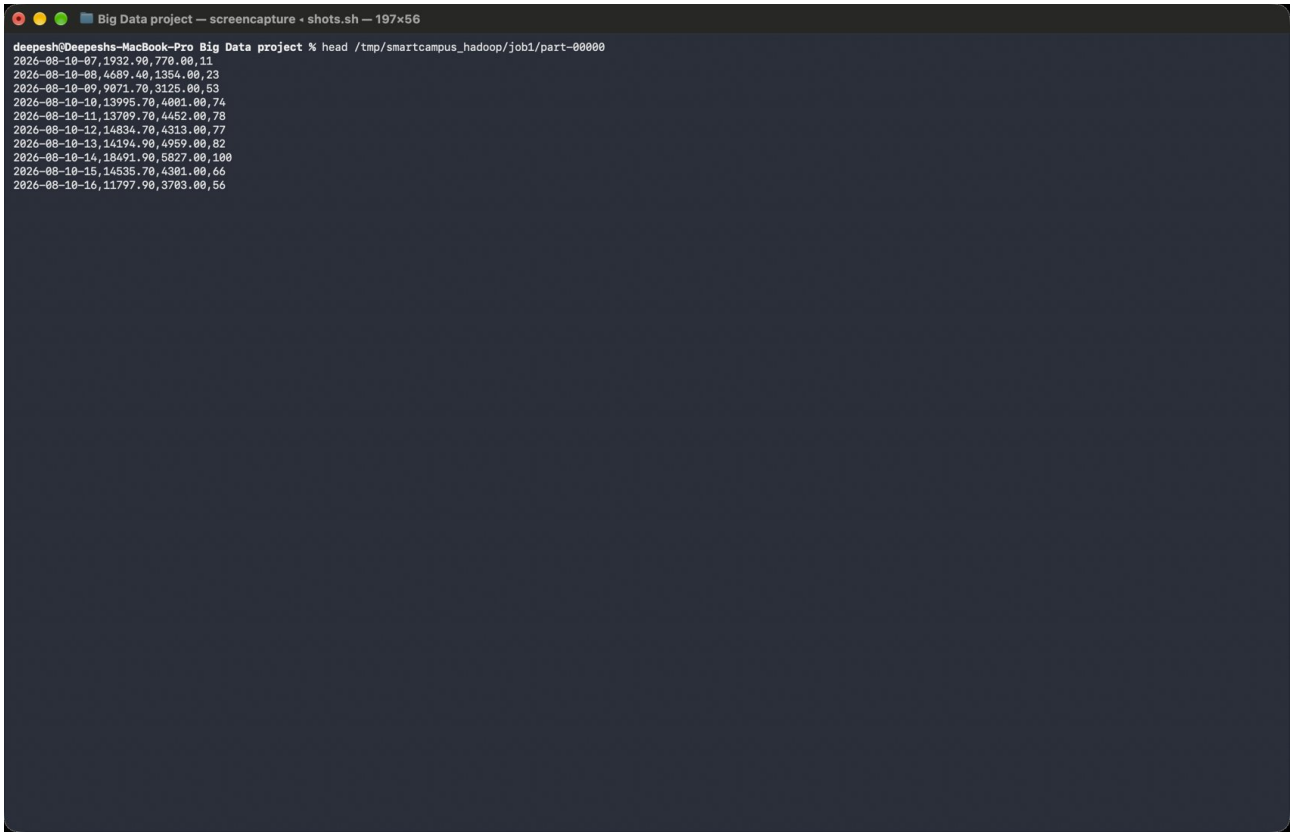


Figure 12. A Hadoop part file (Job 1 output written by Hadoop).

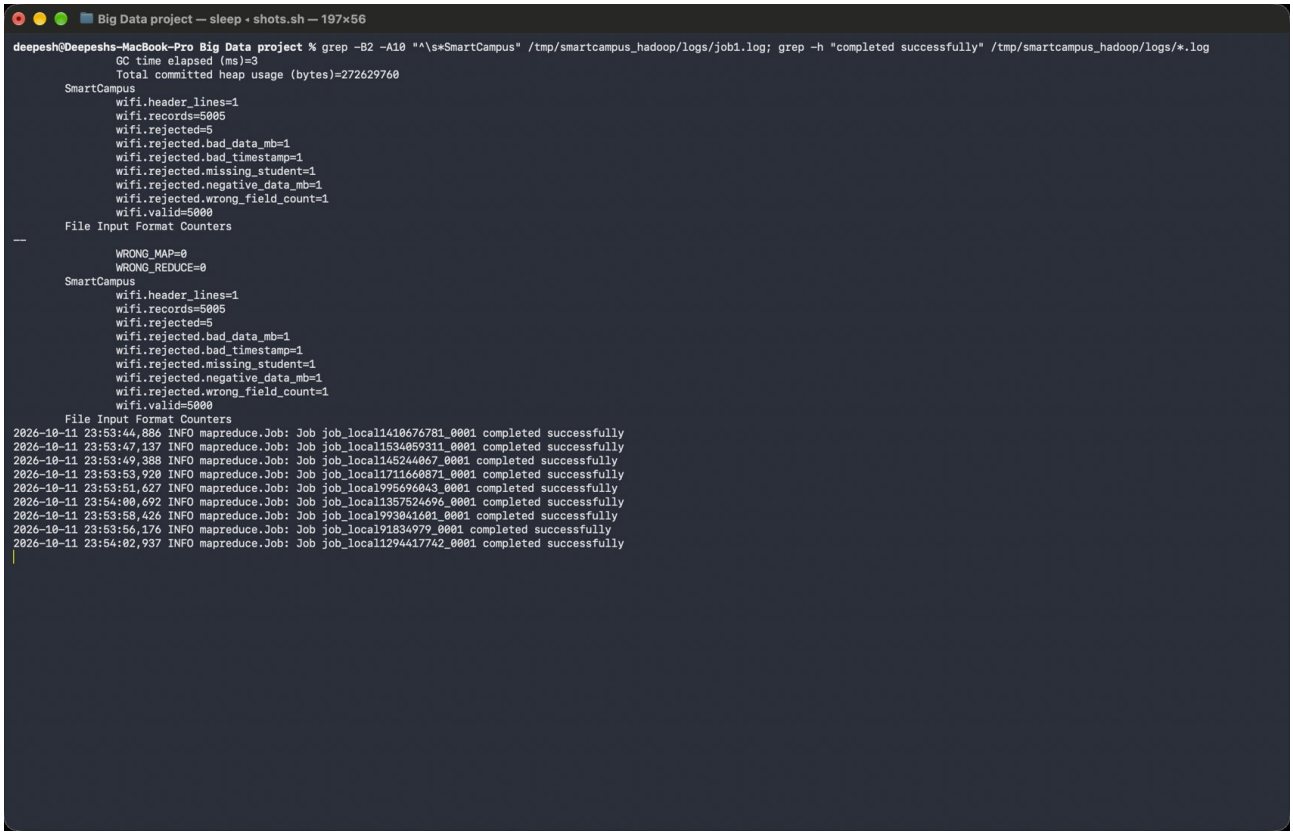


Figure 13. Hadoop counters: SmartCampus rejection counts, and nine jobs completed successfully.

```
$ scripts/run_hadoop.sh
Using /opt/homebrew/opt/hadoop/libexec/share/hadoop/tools/lib/hadoop-streaming-3.5.0.jar
== job1
== job2
== job3
== job4s1
== job4
== job5s1
== job5g
== job5
== job6
== job7

== Comparison with local outputs/ (trailing TAB stripped)
job1 MATCH (113 lines incl. header)
job2 MATCH (13 lines incl. header)
job3 MATCH (13 lines incl. header)
job4 MATCH (67 lines incl. header)
job5 MATCH (7 lines incl. header)
job6 MATCH (785 lines incl. header)
job7 MATCH (67 lines incl. header)
Trailing TAB on reducer output lines: YES (review finding F6 confirmed)
```

Figure 14. Hadoop 3.5.0 re-run on 18 October 2026: all ten stages completed and all seven outputs MATCH the local results, including the map-only Job 7.